Laboratório 07:

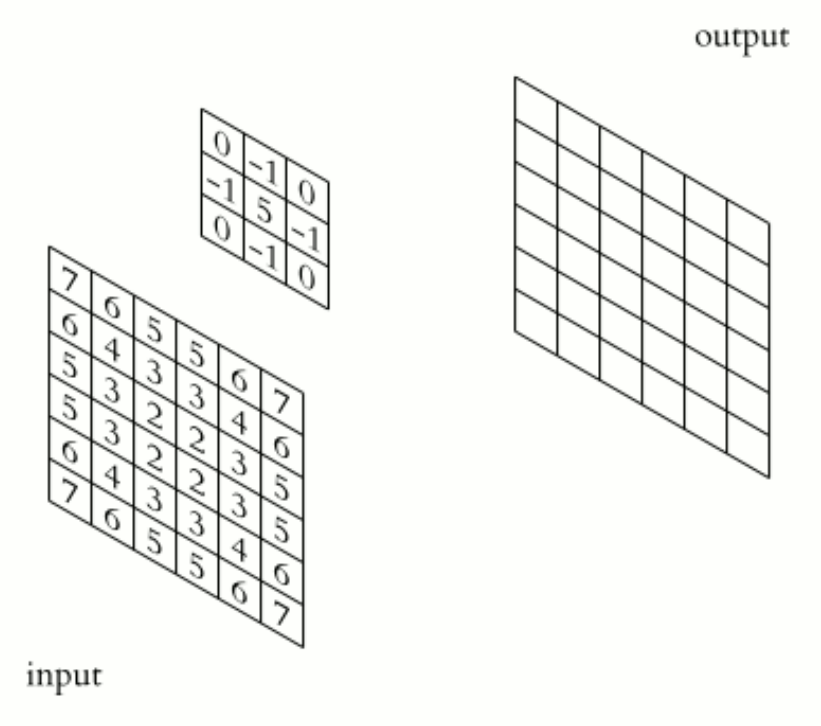
Redes Neurais Convolucionais

CC7711 - Inteligência Artificial

Prof. Dr. Murillo | Depto. de Ciência da Computação

Convolução

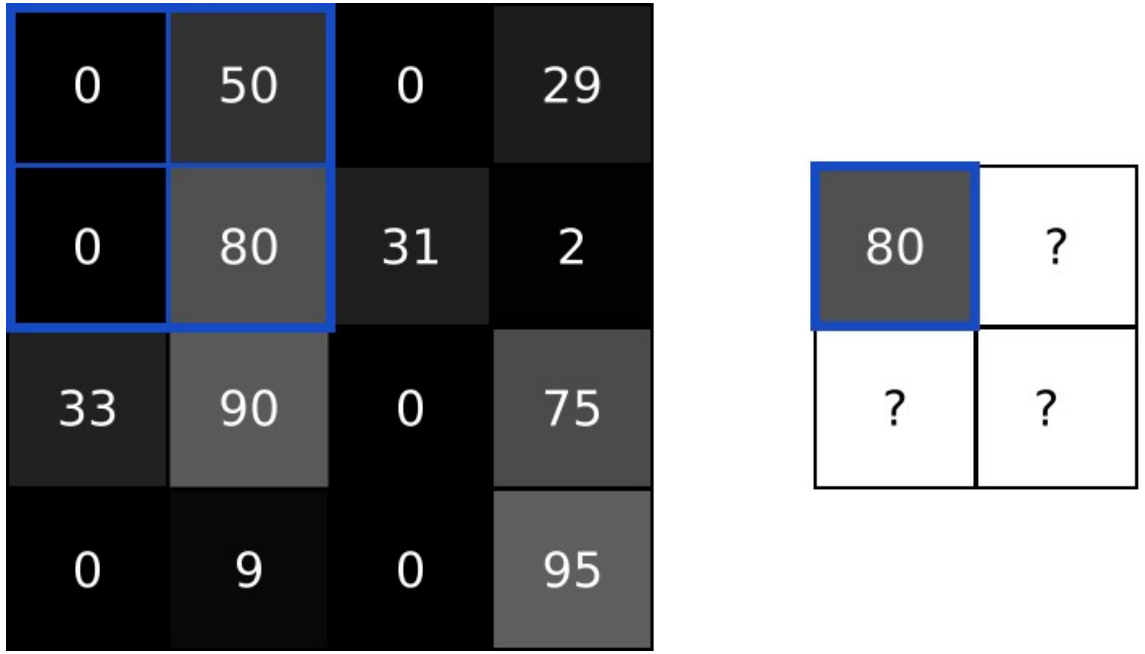
$$(f * g)(k) = h(k) = \sum_{j=0}^{k} f(j) \cdot g(k-j)$$



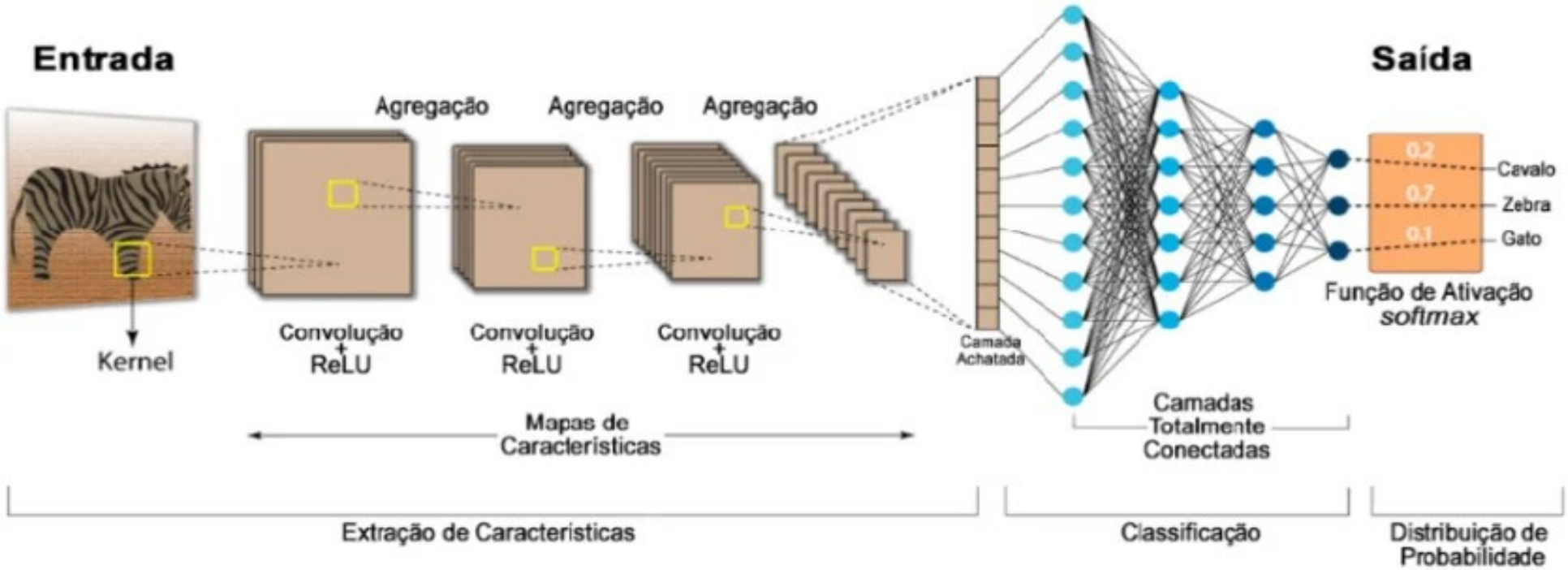
- **f:** Matriz de Entrada.
- **g:** Kernel.
- **h:** Matriz de saída

Max pooling

- **Janela Deslizante:** Uma janela (geralmente 2x2) desliza sobre a matriz de entrada
- **Seleção:** Apenas o valor máximo da janela é selecionado
- **Redução de tamanho:** A matriz de saída é reduzida em 75% com uma janela 2x2



Rede Neural Convolucional



- **Camadas Convolucionais:** Kernels deslizam sobre a imagem para detectar padrões e extrair características.
- **Camadas de Pooling:** Reduz a dimensionalidade dos dados.
- **Camadas Flatten:** Achata as características extraídas para uma única dimensão.
- **Camadas Totalmente Conectadas:** Classificador Multi-Layer Perceptron.

Scikit-learn

TensorFlow + Keras

```
from tensorflow.keras import layers, models
model_cnn = models.Sequential([
      layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
      layers.MaxPooling2D((2, 2)),
      layers.Flatten(),
            layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])
```

```
model_cnn.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
model_cnn.summary()
hist_cnn = model_cnn.fit(x_train_cnn, y_train_m, epochs=30, validation_split=0.1, verbose=1)
```

Desafio 1: MNIST Dataset

Comparar o desempenho da MLP e da CNN utilizando o Keras para reconhecer diferentes digitos da base de dados MNIST

```
import tensorflow as tf
mnist = tf.keras.datasets.mnist
(x_train_m, y_train_m), (x_test_m, y_test_m) = mnist.load_data()
# Normalização base (valores entre 0 e 1)
x_train_m = x_train_m.astype('float32') / 255.0
x_test_m = x_test_m.astype('float32') / 255.0
```

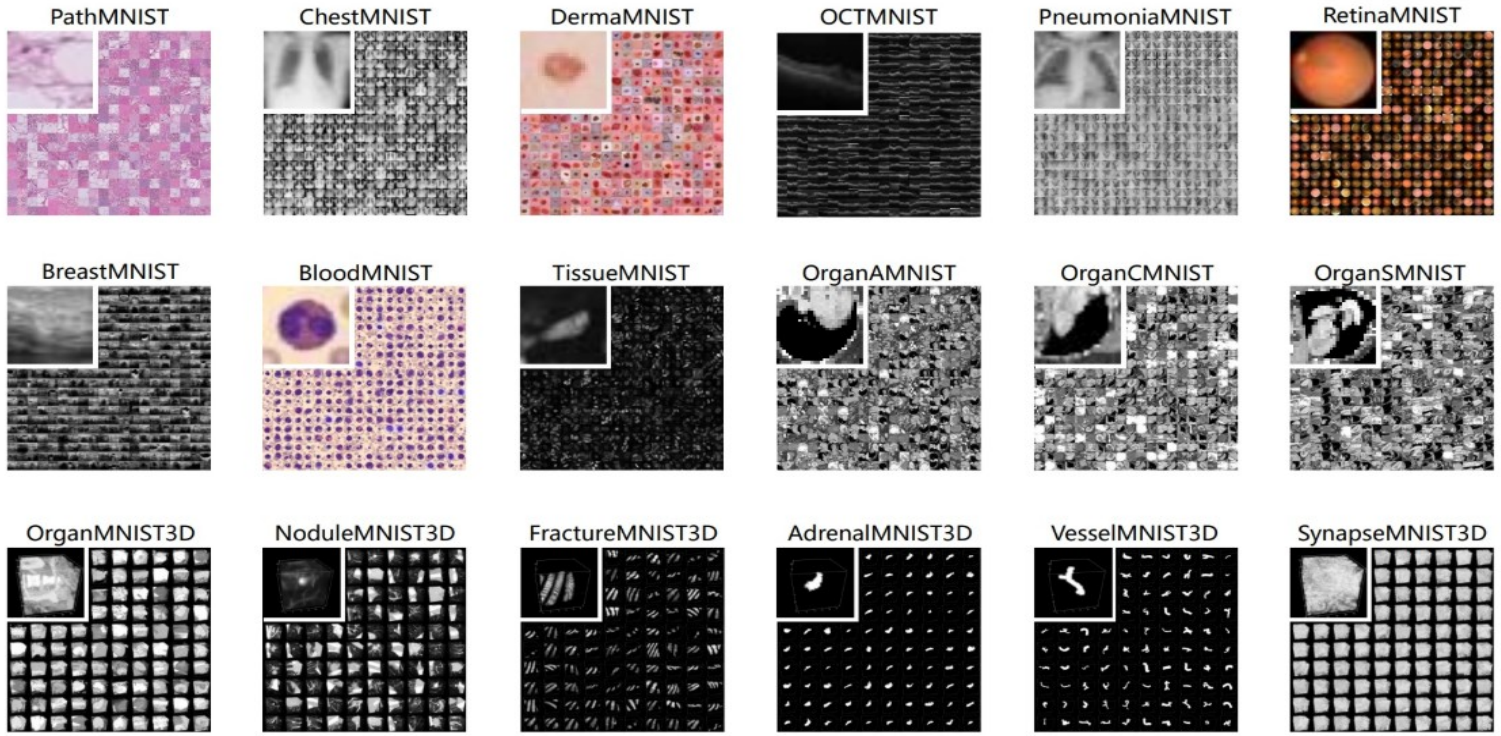
Desafio 2: Cifar-10 Dataset

Utilizar CNN com o Keras para classificar as 10 classes da base CIFAR-10 :

```
import tensorflow as tf
cifar10 = tf.keras.datasets.cifar10
(x_train_c, y_train_c), (x_test_c, y_test_c) = cifar10.load_data()
# Normalização de Tensores RGB
x_train_c, x_test_c = x_train_c / 255.0, x_test_c / 255.0
```

Desafio 3: MedMNIST

Escolher um dos datasets disponíveis do **MedMNIST** e faça a classificação utilizando CNN com o Keras

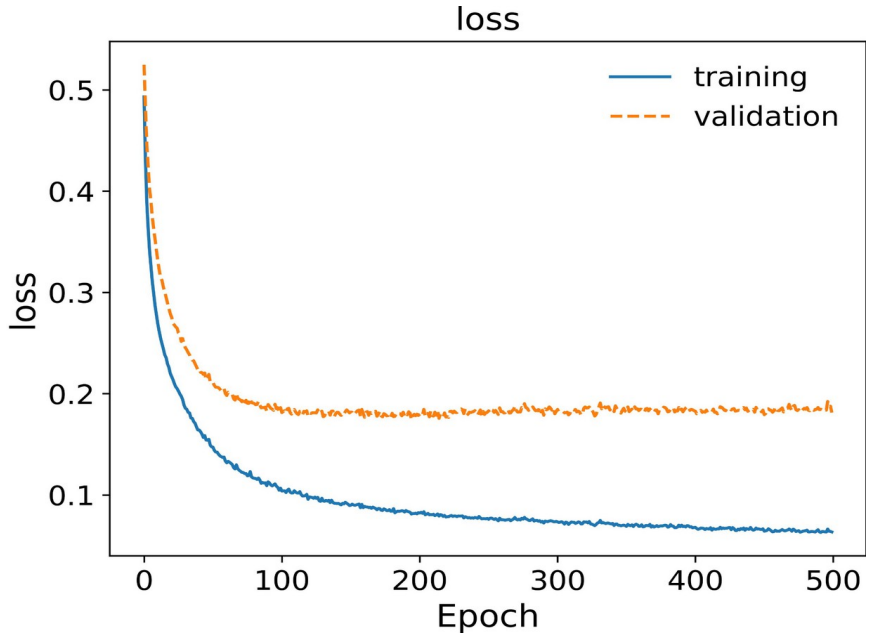



```
!pip install medmnist -q
import medmnist
from medmnist import INFO
data_flag = NOME_DA_BASE_DA_MEDMNIST
info = INFO[data_flag]
DataClass = getattr(medmnist, info['python_class'])
train_dataset = DataClass(split='train', download=True)
test_dataset = DataClass(split='test', download=True)
```

Gráfico de Convergência de Validação

Monitorar o valor de fitness com os dados de validação para cada iteração do treinamento.

- **Gráfico de Convergência:** Evolução do Melhor Fitness vs Épocas.
- **Validação:** Parte do conjunto de dados, separada do conjunto de treinamento, para avaliar o desempenho de um modelo de aprendizado de máquina durante o treinamento
- **Classification Report:** Calcula as principais métricas de avaliação do modelo treinado.




```
from sklearn.metrics import classification_report
y_pred_mlp_probs = model_mlp.predict(x_test_m)
y_pred_mlp = np.argmax(y_pred_mlp_probs, axis=1)
print(classification_report(y_test_m, y_pred_mlp, digits=4))

import matplotlib.pyplot as plt
plt.figure(figsize=(10, 6))
plt.plot(history.history['loss'], label='Treino')
plt.plot(history.history['val_loss'], label='Validação')
plt.show()
```

Roteiro do Laboratório & Entregas

Completem as seguintes tarefas:

- **Passo 1:** Comparar a classificação da base da MNIST utilizando MLP e CNN com o Keras
- **Passo 2:** Classificar a base da CIFAR-10 utilizando CNN com o Keras
- **Passo 3:** Classificar um dos datasets da MedMNIST utilizando CNN com o Keras

Métricas e Submissão: Submetam o notebook com o código completo. Ao final, apresentem a solução dos 3 problemas com as métricas: **Classification Report e Gráfico de Convergência.** **(Obs: Utilizem a GPU do GoogleColab e tentem treinar cada modelo por pelo menos 100 épocas)**